Longformer : The Long-Document Transformer

Beltagy, Iz, Matthew E. Peters, and Arman Cohan. "Longformer: The long-document transformer." *arXiv preprint arXiv:2004.05150* (2020).

Lee Jiwon

PNU Machine Learning & Bioinformatics Lab

목차

1. Introduction
2. Longformer attention mechanism
3. Longformer implementation
4. Autoregressive language modeling
5. Pretraining & finetuning
6. Evaluation & result
7. Longformer Encoder Decoder
8. Conclusion

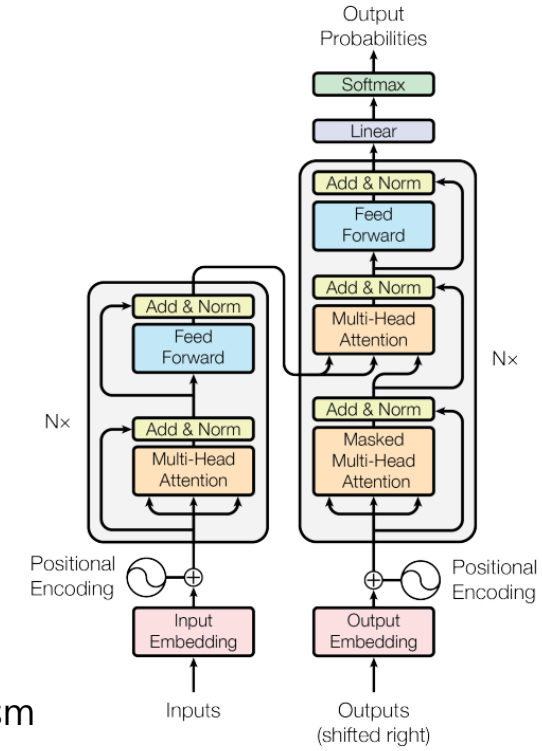
Introduction: Why was Longformer introduced?

Transformer Model Advantages

Achieves **state-of-the-art (SOTA)** performance in various NLP tasks, largely due to **self-attention**.

Transformer Model Limitations

- **Quadratic complexity $O(n^2)$**
- **512-token limit**



Goals of Longformer

1. **$O(n)$ self-attention** to handle long documents (classification, QA, coreference resolution) efficiently
2. **replace the standard self-attention** in pretrained Transformers with Longformer's attention mechanism

Introduction: Why was Longformer introduced?

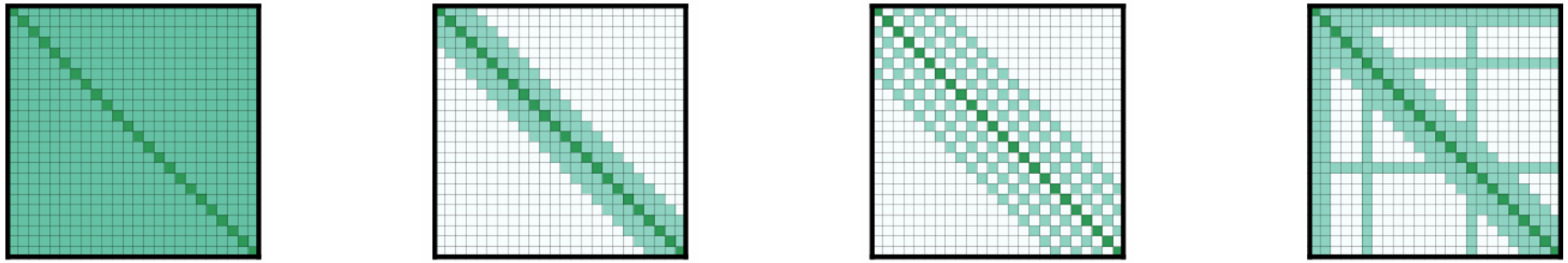
Longformer Attention Mechanism

windowed local-context self-attention + global attention

- **Local attention:**
  Constructs contextual representations within a fixed local window
- **Global attention:**
  Aggregates global sequence information to enhance task-specific predictions, incorporating inductive biases relevant to the task

Longformer : attention pattern

In standard Transformer models, the self-attention complexity is $\boldsymbol{O(n^2)}$
→ **Sparsify** the full self attention matrix !!



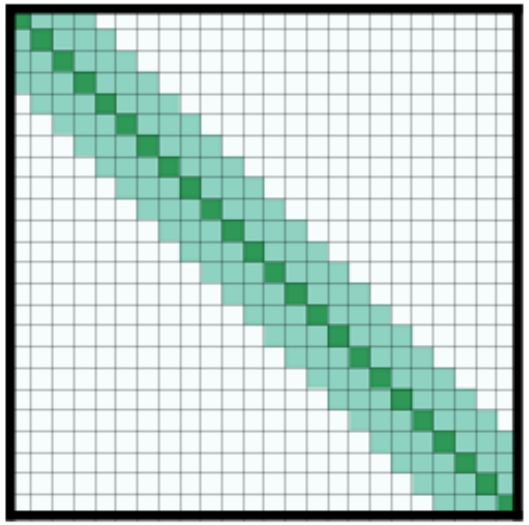
(a) Full n^2 attention (b) Sliding window attention (c) Dilated sliding window (d) Global+sliding window

Figure 2: Comparing the full self-attention pattern and the configuration of attention patterns in our Longformer.

Longformer : attention pattern

Sliding Window Attention

- Designed to effectively capture **local context**
- Computational complexity: $O(n)$
- Each token applies windowed attention with a fixed size (w).
- Each token attends to w/2 tokens on both sides.
- Receptive field size at the top layer: L × W
- **Stacking multiple layers allows higher layers to access the entire sequence**.

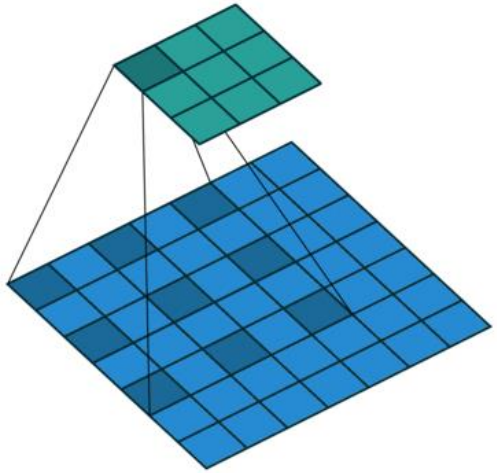


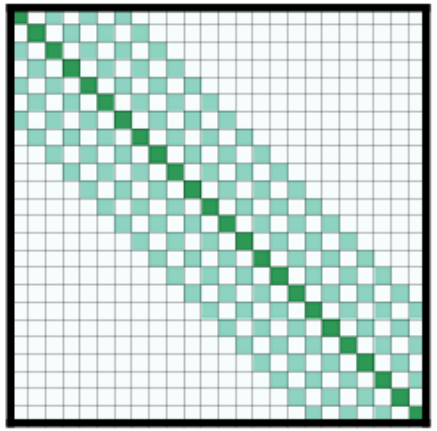
(b) Sliding window attention

Longformer : attention pattern

Dilated Sliding Window Attention

- **What is dilation?**
  Concept from CNNs (Dilated CNN, van den Oord et al., 2016)
  gaps between kernel elements.
- **How does it apply to attention?**
  Adds **d-sized gaps** within the attention window.
  Receptive field size at the top layer: L x d × w.
- **How is dilated attention applied across multihead attention?**
  Different dilation configurations per head improves performance by allowing some heads without dilation to focus on local context, while others with dilation focus on longer context

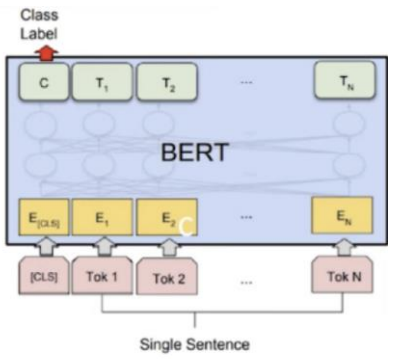




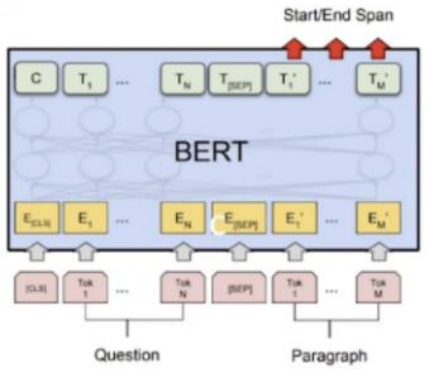
(c) Dilated sliding window

Global Attention

- **Input Representation in BERT-style Models**
  During fine-tuning, the optimal input representation depends on the task.
  Examples
  - ❖ Classification: The entire sequence representation must include a **special CLS token**.
  - ❖ QA (Question Answering): The question and document must be **concatenated** into a single sequence.

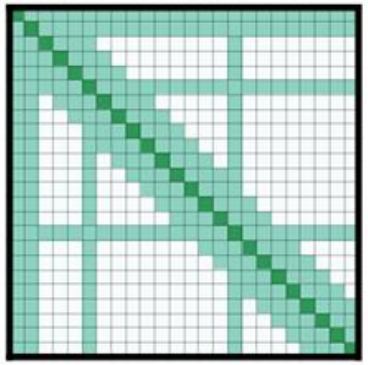


(b) Single Sentence Classification Tasks: SST-2, CoLA



(c) Question Answering Tasks: SQuAD v1.1

- **Why Introduce Global Attention?**
  - for **flexibility** in learning task-specific representations.
  - Tokens with global attention can attend to **all tokens** in the sequence.
    At the same time, all tokens can also reference global attention tokens.
  - Effect: Certain tokens can consider the entire document context.
    - ❖Classification: Global attention is applied to the CLS token.
    - ❖QA: Global attention is applied to all question tokens.

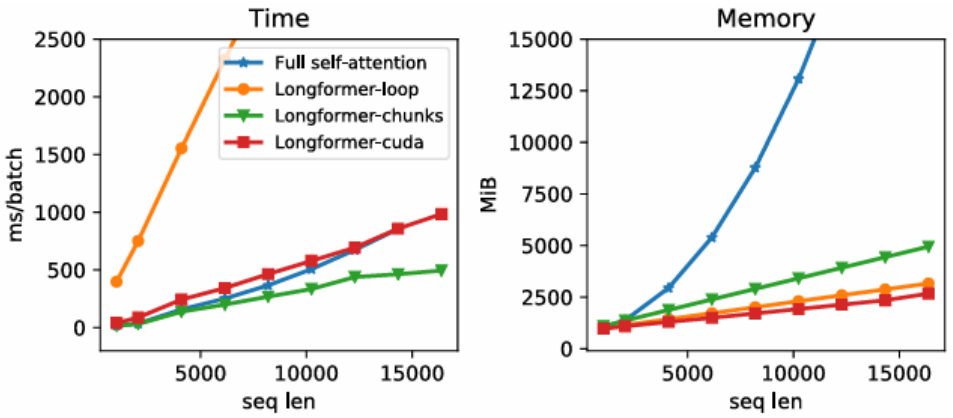

(d) Global+sliding window

Longformer : Implementation

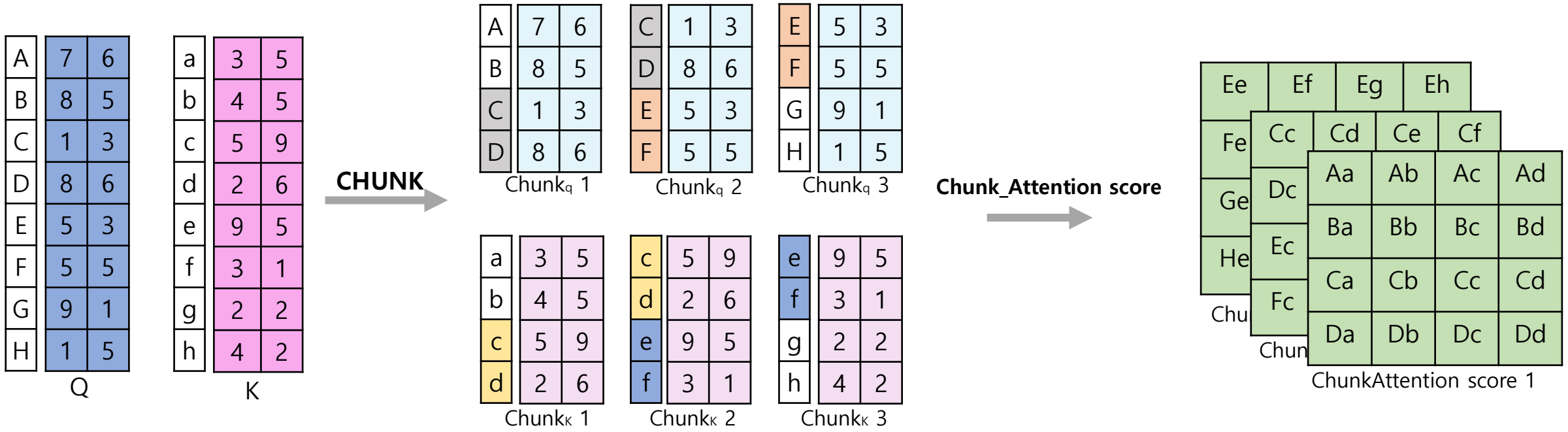
Attention mechanism in longformer
Longformer maintains the overall structure of transformer, modifying only attention mechanism.

```
class Longformer(RobertaModel):
    def __init__(self, config):
        super(Longformer, self).__init__(config)
        if config.attention_mode == 'n2':
            pass  # do nothing, use BertSelfAttention instead
        else:
            for i, layer in enumerate(self.encoder.layer):
                layer.attention.self = LongformerSelfAttention(config, layer_id=i)
```

Implementation Methods
1. Loop-based implementation
2. Chunk-based implementation
3. CUDA acceleration

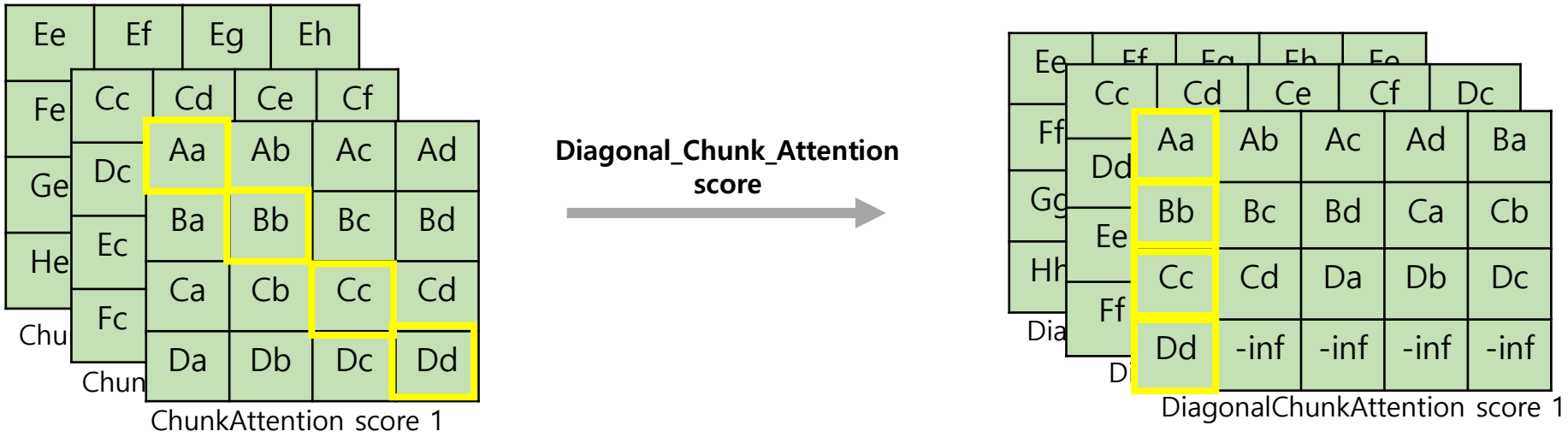


Chunk-based



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Chunk-based


Ee Ef Eg Eh
Fe
Ge
He
Chu
Cc Cd Ce Cf
Dc
Ec
Fc
Chun
Aa Ab Ac Ad
Ba Bb Bc Bd
Ca Cb Cc Cd
Da Db Dc Dd
ChunkAttention score 1
Diagonal_Chunk_Attention score
Ee
Ff
Gg
Hh
Dia
Cc Cd Ce Cf Dc
Dd
Ee
Ff
D
Aa Ab Ac Ad Ba
Bb Bc Bd Ca Cb
Cc Cd Da Db Dc
Dd -inf -inf -inf -inf
DiagonalChunkAttention score 1

Chunk-based

Aa	Ab	Ac	Ad	Ba
Bb	Bc	Bd	Ca	Cb
Cc	Cd	Da	Db	Dc
Dd	-inf	-inf	-inf	-inf

DiagonalChunkAttention score 1

Cc	Cd	Ce	Cf	Dc
Dd	De	Df	Ec	Ed
Ee	Ef	Fc	Fd	Fe
Ff	-inf	-inf	-inf	-inf

DiagonalChunkAttention score 2

Ee	Ef	Eg	Eh	Fe
Ff	Fg	Fh	Ge	Gf
Gg	Gh	He	Hf	Hg
Hh	-inf	-inf	-inf	-inf

DiagonalChunkAttention score 3

		Aa	Ab	Ac
		Bb	Bc	Bd
		Cc	Cd	Ce
		Dd	De	Df
		Ee	Ef	Eg
		Ff	Fg	Fh

Attention matrix (diagonal_attn)

diagonal_attn[:, :-1, :, w:] = diagonal_chunk_attn[:, :, :w, :w + 1]

Longformer : Implementation

Chunk-based

Aa	Ab	Ac	Ad	Ba
Bb	Bc	Bd	Ca	Cb
Cc	Cd	Da	Db	Dc
Dd	-inf	-inf	-inf	-inf

DiagonalChunkAttention score 1

Cc	Cd	Ce	Cf	Dc
Dd	De	Df	Ec	Ed
Ee	Ef	Fc	Fd	Fe
Ff	-inf	-inf	-inf	-inf

DiagonalChunkAttention score 2

Ee	Ef	Eg	Eh	Fe
Ff	Fg	Fh	Ge	Gf
Gg	Gh	He	Hf	Hg
Hh	-inf	-inf	-inf	-inf

DiagonalChunkAttention score 3

		Aa	Ab	Ac
		Bb	Bc	Bd
		Cc	Cd	Ce
		Dd	De	Df
		Ee	Ef	Eg
		Ff	Fg	Fh
		Gg	Gh	He
		Hh	-inf	-inf

Attention matrix (diagonal_attn)

diagonal_attn[:, -1, :, w:] = diagonal_chunk_attn[:, -1, w:, :w + 1]

Longformer : Implementation

Chunk-based

Aa	Ab	Ac	Ad	Ba
Bb	Bc	Bd	Ca	Cb
Cc	Cd	Da	Db	Dc
Dd	-inf	-inf	-inf	-inf

DiagonalChunkAttention score 1

Cc	Cd	Ce	Cf	Dc
Dd	De	Df	Ec	Ed
Ee	Ef	Fc	Fd	Fe
Ff	-inf	-inf	-inf	-inf

DiagonalChunkAttention score 3

Ee	Ef	Eg	Eh	Fe
Ff	Fg	Fh	Ge	Gf
Gg	Gh	He	Hf	Hg
Hh	-inf	-inf	-inf	-inf

DiagonalChunkAttention score 2

		Aa	Ab	Ac
		Bb	Bc	Bd
Ca	Cb	Cc	Cd	Ce
Db	Dc	Dd	De	Df
Ec	Ed	Ee	Ef	Eg
Fd	Fe	Ff	Fg	Fh
Ge	Gf	Gg	Gh	He
Hf	Hg	Hh	-inf	-inf

Attention matrix (diagonal_attn)

diagonal_attn[:, 1:, :, :w] = diagonal_chunk_attn[:, :, - (w + 1):-1, w + 1:]

Chunk-based

Aa	Ab	Ac	Ad	Ba
Bb	Bc	Bd	Ca	Cb
Cc	Cd	Da	Db	Dc
Dd	-inf	-inf	-inf	-inf

DiagonalChunkAttention score 1

Cc	Cd	Ce	Cf	Dc
Dd	De	Df	Ec	Ed
Ee	Ef	Fc	Fd	Fe
Ff	-inf	-inf	-inf	-inf

DiagonalChunkAttention score 2

Ee	Ef	Eg	Eh	Fe
Ff	Fg	Fh	Ge	Gf
Gg	Gh	He	Hf	Hg
Hh	-inf	-inf	-inf	-inf

DiagonalChunkAttention score 3

		Aa	Ab	Ac
	Ba	Bb	Bc	Bd
Ca	Cb	Cc	Cd	Ce
Db	Dc	Dd	De	Df
Ec	Ed	Ee	Ef	Eg
Fd	Fe	Ff	Fg	Fh
Ge	Gf	Gg	Gh	He
Hf	Hg	Hh	-inf	-inf

Attention matrix (diagonal_attn)

diagonal_attn[:, 0, 1:w, 1:w] = diagonal_chunk_attn[:, 0, :w - 1, 1 - w:]

Chunk-based

Aa	Ab	Ac	Ad	Ba
Bb	Bc	Bd	Ca	Cb
Cc	Cd	Da	Db	Dc
Dd	-inf	-inf	-inf	-inf

DiagonalChunkAttention score 1

Cc	Cd	Ce	Cf	Dc
Dd	De	Df	Ec	Ed
Ee	Ef	Fc	Fd	Fe
Ff	-inf	-inf	-inf	-inf

DiagonalChunkAttention score 2

Ee	Ef	Eg	Eh	Fe
Ff	Fg	Fh	Ge	Gf
Gg	Gh	He	Hf	Hg
Hh	-inf	-inf	-inf	-inf

DiagonalChunkAttention score 3

-inf	-inf	Aa	Ab	Ac
-inf	Ba	Bb	Bc	Bd
Ca	Cb	Cc	Cd	Ce
Db	Dc	Dd	De	Df
Ec	Ed	Ee	Ef	Eg
Fd	Fe	Ff	Fg	Fh
Ge	Gf	Gg	Gh	-inf
Hf	**Hg**	**Hh**	**Hi**	**Hj**

Final attention matrix (slinding window attention output)

diagonal_attn[:, 0, 1:w, 1:w] = diagonal_chunk_attn[:, 0, :w - 1, 1 - w:]

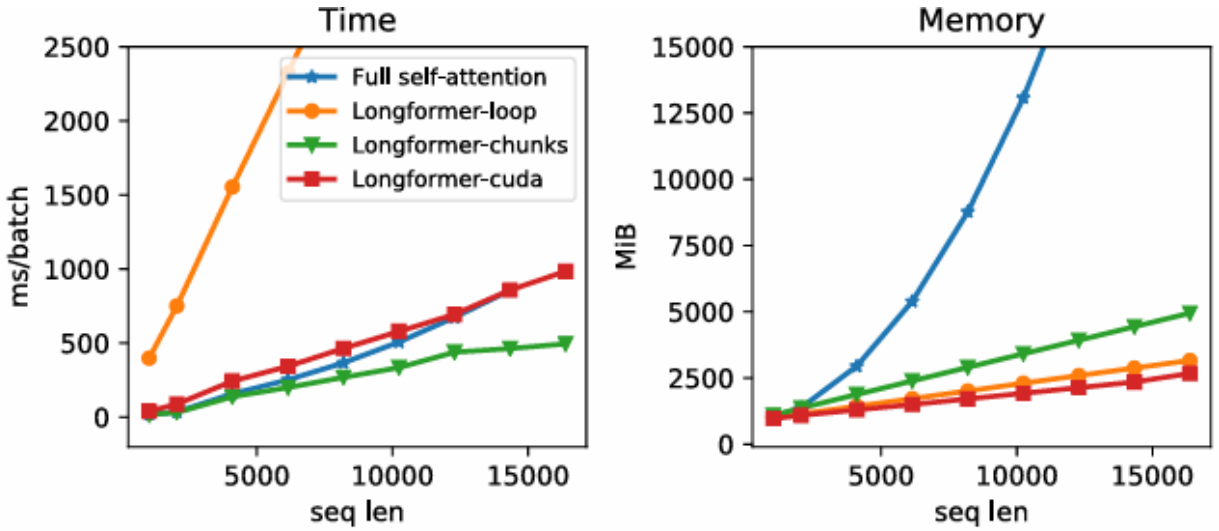
Longformer : Implementation

- **Chunk-based Computation**
  Similar to Q, K calculations, the V and attention output are computed in chunks, generating the final context vector

$$Q_s, K_s, V_s \qquad Q_g, K_g, V_g$$

- **Advantages of Chunk-based Computation**
  1. Utilizes **PyTorch's batched matrix-matrix multiplication** to significantly improve computation speed.
  2. Highly **computationally efficient**, making it well-suited for **pretraining and fine-tuning**.

- **Disadvantages of Chunk-based Computation**
  Memory usage doubles, but this increase was not a major issue during pretraining and fine-tuning.



Flow of the Longformer Paper

1. **Autoregressive Language Modeling**
   Demonstrates Longformer's effectiveness in modeling long documents

2. **Pretrained Model & Fine-tuning**
   Investigates whether Longformer is suitable for long document tasks.
   Pretrained longformer is fine-tuned for specific tasks like classification, QA, and coreference resolution

3. **Encoder-Decoder Model**
   Examines Longformer's performance on sequence-to-sequence (seq2seq) tasks

Autoregressive language modeling

- Based on **Transformer-XL**
- Developed by modifying the implementation of Transformer-XL (Dai et al., 2019)
- Attention pattern: **Dilated sliding window attention**

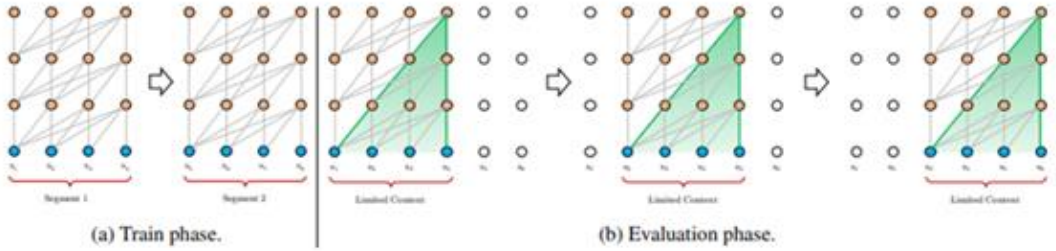


Figure 1: Illustration of the vanilla model with a segment length 4.

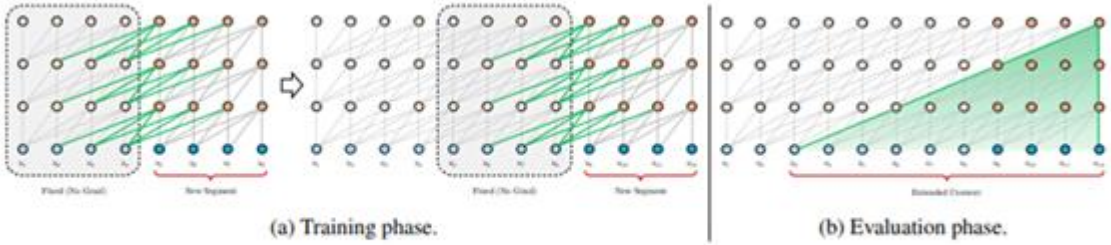
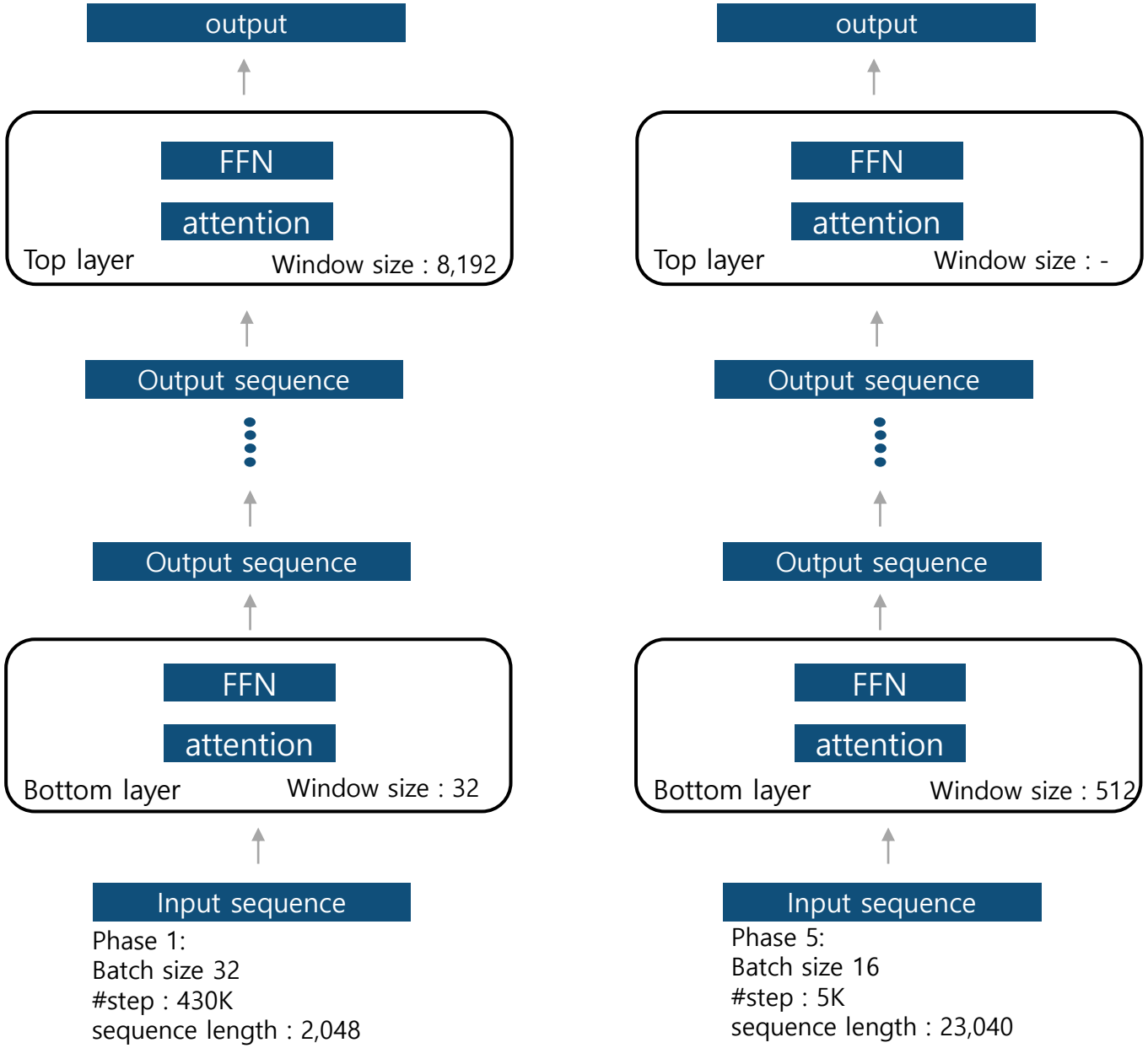


Figure 2: Illustration of the Transformer-XL model with a segment length 4.

Autoregressive language modeling



Param	Value
Position Embeddings	Relative and Sinusoidal as in Dai et al. (2019)
Small model config	12 layers, 8 heads, 512 hidden size as in Dai et al. (2019)
Large model config	30 layers, 8 heads, 512 hidden size as in Child et al. (2019)
Optimizer	AdamW
Dropout	0.2 (small model), 0.4 (large model)
Gradient clipping	0.25
Weight Decay	0.01
Layernorm Location	pre-layernorm (Xiong et al., 2020)
Activation	GeLU
Number of phases	5
Phase 1 window sizes	32 (bottom layer) - 8,192 (top layer)
Phase 5 window sizes	512 (bottom layer) - (top layer)
Phase 1 sequence length	2,048
Phase 5 sequence length	23,040 (gpu memory limit)
Phase 1 LR	0.00025
Phase 5 LR	000015625
Batch size per phase	32, 32, 16, 16, 16
#Steps per phase (small)	430K, 50k, 50k, 35k, 5k
#Steps per phase (large)	350K, 25k, 10k, 5k, 5k
Warmup	10% of the phase steps with maximum 10K steps
LR scheduler	constant throughout each phase
Dilation (small model)	0 (layers 0-5), 1 (layers 6-7), 2 (layers 8-9), 3 (layers 10-11)
Dilation (large model)	0 (layers 0-14), 1 (layers 15-19), 2 (layers 20-24), 3 (layers 25-29)
Dilation heads	2 heads only

Number of gradient updates / phase

Table 12: Hyperparameters for the best performing model for character-level language modeling

Autoregressive language modeling

Training, test Dataset: text8, enwik8

100M characters from Wikipedia split into 90M, 5M, 5M

Evaluation Method : BPC (Bits per character)

Measures the average number of bits needed to encode a character.
Lower BPC indicates better performance.

Result (Table 2, 3)

small model : longformer acheives a new SOTA on text8, enwik8
Large model : longformer achieves outperform or slightly underperforms recent models tha have mofr than twice the number of parameters on enwik8.

Model	#Param	Dev	Test
Dataset `text8`			
T12 (Al-Rfou et al., 2018)	44M	-	1.18
Adaptive (Sukhbaatar et al., 2019)	38M	1.05	1.11
BP-Transformer (Ye et al., 2019)	39M	-	1.11
Our Longformer	41M	1.04	**1.10**
Dataset `enwik8`			
T12 (Al-Rfou et al., 2018)	44M	-	1.11
Transformer-XL (Dai et al., 2019)	41M	-	1.06
Reformer (Kitaev et al., 2020)	-	-	1.05
Adaptive (Sukhbaatar et al., 2019)	39M	1.04	1.02
BP-Transformer (Ye et al., 2019)	38M	-	1.02
Our Longformer	41M	1.02	**1.00**

Table 2: *Small* model BPC on `text8` & `enwik8`

Model	#Param	Test BPC
Transformer-XL (18 layers)	88M	1.03
Sparse (Child et al., 2019)	≈100M	0.99
Transformer-XL (24 layers)	277M	0.99
Adaptive (Sukhbaatar et al., 2019)	209M	0.98
Compressive (Rae et al., 2020)	277M	0.97
Routing (Roy et al., 2020)	≈223M	0.99
Our Longformer	102M	0.99

Table 3: Performance of *large* models on `enwik8`

Ablation study (Table 4)

1. **Window Size** Comparison
   Gradually <u>increasing window size</u> across layers leads to the best performance.
2. **Dilation** Comparison
   Using <u>dilation</u> on 2 heads <u>improves</u> performance compared to no dilation

Model	Dev BPC
Decreasing w (from 512 to 32)	1.24
Fixed w (= 230)	1.23
Increasing w (from 32 to 512)	**1.21**
No Dilation	1.21
Dilation on 2 heads	**1.20**

Table 4: Top: changing window size across layers. Bottom: with/without dilation (@ 150K steps on phase1)

Flow of the Longformer Paper

1. **Autoregressive Language Modeling**
   Demonstrates Longformer's effectiveness in modeling long documents

2. **Pretrained Model & Fine-tuning**
   Investigates whether Longformer is suitable for long document tasks.
   Pretrained longformer is fine-tuned for specific tasks like classification, QA, and coreference resolution

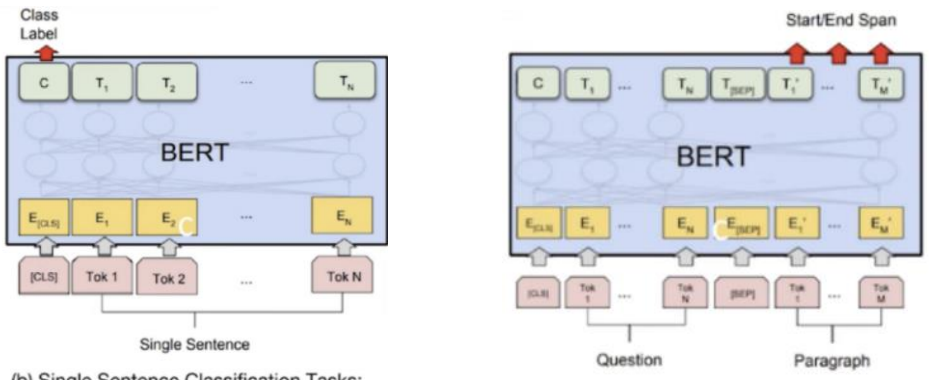
3. **Encoder-Decoder Model**
   Examines Longformer's performance on sequence-to-sequence (seq2seq) tasks

Pretraining & Finetuning

- **Overview**
  Most modern NLP systems use pretrained models, which are then fine-tuned for specific tasks (e.g., BERT).

- **Goal**
  1. Develop a **pretrained model optimized for long-document processing**.
  2. Perform tasks such as **Classification, QA, and Coreference Resolution**.

- **Pretraining Attention Pattern**
  Sliding window attention (window size = 512).



(b) Single Sentence Classification Tasks: SST-2, CoLA

(c) Question Answering Tasks: SQuAD v1.1

Pretraining Method

- **Based on RoBERTa :**
  Longformer continues training from **RoBERTa's checkpoint** using Masked Language Modeling (MLM).
- **Training Data :**
  A corpus of long documents, including datasets from the original RoBERTa.
- **Sequence length :**
  4,096 tokens

RoBERTa's position embeddings are trained for 512 tokens, while Longformer extends this to 4,096 tokens

Copy Initialization: RoBERTa's 512 position embeddings are copied multiple times to extend them to 4,096 tokens

Source	Tokens	Avg doc len
Books (Zhu et al., 2015)	0.5B	95.9K
English Wikipedia	2.1B	506
Realnews (Zellers et al., 2019)	1.8B	1.7K
Stories (Trinh and Le, 2018)	2.1B	7.8K

Table 13: Pretraining data

Pretraining Results (Table 5)

1. **RoBERTa's performance is comparable** to its original paper (BPC: 1.880), indicating similar data distribution
2. Sliding window attention works efficiently, enabling better utilization of long contexts.

Model	base	large
RoBERTa (seqlen: 512)	1.846	1.496
Longformer (seqlen: 4,096)	10.299	8.738
+ copy position embeddings	1.957	1.597
+ 2K gradient updates	1.753	1.414
+ 65K gradient updates	1.705	1.358
Longformer (train extra pos. embed. only)	1.850	1.504

Table 5: MLM BPC for RoBERTa and various pre-trained Longformer configurations.

Frozen RoBERTa Weights (Weight-Freezing Experiment)

- **Objective:**
  What happens if we retain RoBERTa's performance while only learning new position embeddings?

  Results:
  Only training position embeddings → BPC: 1.85
  Training all weights → BPC: 1.70 (better performance)

Finetuning

- **Objective**
  1. Can **Longformer's attention mechanism** replace **BERT's self-attention**?
  2. Can it **overcome BERT's 512-token limitation**?

- **Task 1: Question Answering (QA)**
  Datasets: WikiHop, TriviaQA, HotpotQA

- **Task 2: Coreference Resolution**
  Dataset: OntoNotes

- **Task 3: Document Classification**
  Datasets: IMDB, Hyperpartisan News Detection

Result

Summary of all finetuning experiments (Table 7)

Longformer **outperform**s RoBERTa across all tasks
Particularly strong improvements in long-context tasks such as WikiHop and Hyperpartisan.

Analysis by Task

1. **TriviaQA:**
   Many answers are found in local contexts near the question, reducing the need for long-context processing.
2. **HotpotQA:**
   The model can easily locate relevant documents using supporting facts.
   Since the local context is sufficient, long-context modeling provides limited additional benefits.
3. **WikiHop:**
   Requires reasoning chains rather than simple question-answer matching.
   This necessitates using the entire document, making Longformer's long-context processing highly effective.
4. **IMDB & OntoNotes**
   IMDB → The dataset consists mostly of short documents, so Longformer's long-context processing isn't beneficial.
   OntoNotes → The distance between mentions is short, meaning that traditional coreference resolution methods still perform well without requiring long-range dependencies.

	QA			Coref.	Classification	
Model	WikiHop	TriviaQA	HotpotQA	OntoNotes	IMDB	Hyperpartisan
RoBERTa-base	72.4	74.3	63.5	78.4	95.3	87.4
Longformer-base	**75.0**	**75.2**	**64.4**	**78.6**	**95.7**	**94.8**

Table 7: Summary of finetuning results on QA, coreference resolution, and document classification. Results are on the development sets comparing our Longformer-base with RoBERTa-base. TriviaQA, Hyperpartisan metrics are F1, WikiHop and IMDB use accuracy, HotpotQA is joint F1, OntoNotes is average F1.

Flow of the Longformer Paper

1. **Autoregressive Language Modeling**
   Demonstrates Longformer's effectiveness in modeling long documents

2. **Pretrained Model & Fine-tuning**
   Investigates whether Longformer is suitable for long document tasks.
   Pretrained longformer is fine-tuned for specific tasks like classification, QA, and coreference resolution

3. **Encoder-Decoder Model**
   Examines Longformer's performance on sequence-to-sequence (seq2seq) tasks

Longformer Encoder Decoder

Challenges of Encoder-Decoder Models (Seq2Seq)

Handling long sequence inputs is difficult in traditional seq2seq models.

Introducing Longformer-Based Encoder-Decoder Model:

Encoder : local + global attention
Decoder: full self-attention.

Leveraging Pretrained BART Weights

Uses the same architecture as BART.
Expands position embeddings by copying them **16 times** to handle longer sequences.

Summarization Task Evaluation

- Dataset: arXiv scientific paper summaries (14.5K tokens).
- Model behavior:
  1. Encoder receives the full document
  2. Decoder generates the summary
- Attention Mechanism :
  1. Encoder: Window size = 1024, global attention on<s>token.
  2. Decoder: Uses full attention.
- Performance:
  Outperforms BigBird even without additional pretraining.
  Figure 3: Performance improves as input length increases.

	R-1	R-2	R-L
Discourse-aware (2018)	35.80	11.05	31.80
Extr-Abst-TLM (2020)	41.62	14.69	38.03
Dancer (2020)	42.70	16.54	38.44
Pegasus (2020)	44.21	16.95	38.83
LED-large (seqlen: 4,096) (ours)	44.40	17.94	39.76
BigBird (seqlen: 4,096) (2020)	**46.63**	19.02	41.77
LED-large (seqlen: 16,384) (ours)	**46.63**	**19.62**	**41.83**

Table 11: Summarization results of Longformer-Encoder-Decoder (LED) on the arXiv dataset. Metrics from left to right are ROUGE-1, ROUGE-2 and ROUGE-L.

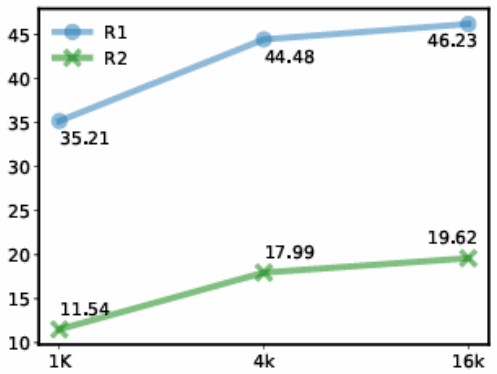


Figure 3: ROUGE-1 and ROUGE-2 of LED when varying the input size (arXiv validation set).

Conclusion

Longformer efficiently processes long documents using a Transformer-based model.
Unlike traditional methods, it handles long sequences without chunking or truncation, leveraging local and global attention for linear scalability.

Performance

1. Longformer achieves **state-of-the-art (SOTA)** performance on character-level language modeling tasks, such as text8 and enwik8.
2. Pretrained Longformer consistently **outperforms** RoBERTa on long-document tasks.
3. It sets new **SOTA** results on WikiHop and TriviaQA.
4. LED achieves **SOTA** performance on long-document summarization, specifically on the arXiv summarization task.

Q&A